

1. Что такое ORM и почему SQLAlchemy считается одним из самых мощных инструментов ORM в Python?

ORM (Object-Relational Mapping)

ORM — технология, позволяющая работать с таблицами БД как с объектами Python.

Основные принципы:

- класс ↔ таблица;
- объект ↔ строка;
- атрибут ↔ столбец;
- связи между объектами ↔ связи между таблицами.

Преимущества SQLAlchemy

- поддержка множества СУБД (SQLite, PostgreSQL, MySQL и др.);
- независимость кода от конкретной БД;
- автоматическая генерация SQL;
- поддержка ORM и SQLAlchemy Core;
- управление транзакциями;
- поддержка сложных запросов и отношений между таблицами.

2. Создание SQLAlchemy Engine

Engine — объект, управляющий подключением к БД.

```
from sqlalchemy import create_engine
```

SQLite

```
engine = create_engine("sqlite:///database.db")
```

PostgreSQL

```
engine = create_engine(
    "postgresql://user:password@localhost/dbname"
)
```

MySQL

```
engine = create_engine(
    "mysql+pymysql://user:password@localhost/dbname"
)
```

3. Session и управление транзакциями

Session

Session — объект для работы с ORM-объектами и транзакциями.

Создание:

```
from sqlalchemy.orm import sessionmaker

Session = sessionmaker(bind=engine)
session = Session()
```

Основные методы

- `add()` — добавить объект;
- `commit()` — сохранить изменения;
- `rollback()` — отменить изменения;
- `close()` — закрыть сессию.

Пример:

```
try:
    session.add(user)
    session.commit()
except:
    session.rollback()
finally:
    session.close()
```

4. Declarative Base

Назначение

Declarative Base — базовый класс, от которого наследуются модели БД.

Позволяет описывать таблицы в виде классов.

Пример:

```
from sqlalchemy.orm import DeclarativeBase
from sqlalchemy import Integer, String
from sqlalchemy.orm import Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String)
```

Создание таблиц:

```
Base.metadata.create_all(engine)
```

5. SQLAlchemy Core и ORM

SQLAlchemy Core

Работа с SQL через Python-конструкции.

Преимущества:

- выше производительность;
- полный контроль над SQL;
- удобно для сложных запросов.

ORM

Работа через объекты Python.

Преимущества:

- меньше кода;
- удобнее разработка;
- автоматическое преобразование объектов.

Когда использовать

Core

- сложные SQL-запросы;
- высокая производительность.

ORM

- большинство CRUD-приложений;
- веб-приложения;
- бизнес-логика.

6. Отношения между таблицами

Один-ко-многим

```
class User(Base):
    __tablename__ = "users"

    id = mapped_column(primary_key=True)
    posts = relationship("Post", backref="user")

class Post(Base):
    __tablename__ = "posts"

    id = mapped_column(primary_key=True)
    user_id = mapped_column(ForeignKey("users.id"))
```

Многие-ко-многим

Через промежуточную таблицу.

```
association = Table(
    "association",
    Base.metadata,
    Column("user_id", ForeignKey("users.id")),
    Column("group_id", ForeignKey("groups.id"))
)
users = relationship("User", secondary=association, backref="groups")
```

Основные параметры

- `relationship()` — описание связи;
- `backref` — автоматическая обратная связь;
- `ForeignKey` — внешний ключ.

7. CRUD-операции

Create

```
user = User(name="Ivan")
session.add(user)
session.commit()
```

Read

```
users = session.query(User).all()

user = session.query(User).filter_by(name="Ivan").first()
```

Update

```
user.name = "Petr"
session.commit()
```

ИЛИ

```
session.query(User).filter_by(id=1).update({"name": "Petr"})
session.commit()
```

Delete

```
session.delete(user)
session.commit()
```

8. Фильтрация, сортировка и Join

Фильтрация

```
session.query(User).filter(User.age > 18)
```

ИЛИ

```
session.query(User).filter_by(name="Ivan")
```

Сортировка

```
session.query(User).order_by(User.name)
```

По убыванию:

```
from sqlalchemy import desc

session.query(User).order_by(desc(User.age))
```

Join

```
session.query(User).join(Post)
```

Можно комбинировать:

```
session.query(User) \
    .join(Post) \
    .filter(Post.title == "SQLAlchemy")
```

9. Выполнение "сырых" SQL-запросов

Используется функция `text()`.

```
from sqlalchemy import text

result = session.execute(
    text("SELECT * FROM users WHERE id=:id"),
    {"id": 1}
)

rows = result.fetchall()
```

Также можно использовать:

```
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM users"))
```

10. Управление транзакциями и обработка исключений

Рекомендуется использовать контекстный менеджер.

```
from sqlalchemy.orm import Session

with Session(engine) as session:
    try:
        session.add(user)
        session.commit()
    except:
        session.rollback()
```

Основные принципы:

- все изменения выполняются внутри транзакции;
- `commit()` — подтверждает изменения;
- `rollback()` — отменяет изменения при ошибке;
- после завершения работы сессия закрывается.

11. Пул соединений (Connection Pool)

Понятие

Пул соединений — набор заранее созданных соединений с БД, которые повторно используются приложением.

Преимущества

- уменьшает время подключения;
- снижает нагрузку на сервер БД;
- повышает производительность;
- эффективно работает при большом количестве запросов.

Настройка

```
engine = create_engine(  
    url,  
    pool_size=10,  
    max_overflow=20  
)
```

Основные параметры:

- `pool_size` — количество постоянных соединений;
- `max_overflow` — дополнительные временные соединения;
- `pool_timeout` — время ожидания свободного соединения;
- `pool_recycle` — время жизни соединения.

12. Стратегии загрузки данных (Lazy/Eager Loading)

Lazy Loading (ленивая загрузка)

Связанные данные загружаются **только при первом обращении** к ним.

```
user.posts
```

Преимущества:

- экономия памяти;
- быстрее первоначальный запрос.

Недостаток:

- возможна проблема **N+1 запросов**.

Eager Loading (жадная загрузка)

Связанные данные загружаются **сразу вместе с основным запросом**.

Основные способы:

- `joinedload()` — через `JOIN`;
- `selectinload()` — отдельным запросом с `IN`;
- `subqueryload()` — через подзапрос.

Пример:

```
from sqlalchemy.orm import joinedload

users = session.query(User).options(joinedload(User.posts)).all()
```

Сравнение

Lazy	Eager
Загружает по требованию	Загружает сразу
Меньше данных в начале	Меньше количества запросов
Возможен N+1	Лучше для связанных данных

13. Работа с Session в веб-приложениях

Принцип

На **каждый HTTP-запрос** создаётся отдельная `Session`, которая закрывается после обработки запроса.

Жизненный цикл:

1. создать `Session`;
2. выполнить запросы;
3. `commit()` или `rollback()`;
4. закрыть `Session`.

Flask

Часто используется `Flask-SQLAlchemy`.

```
db.session.add(user)
db.session.commit()
```

Сессия автоматически привязана к запросу.

Без Flask-SQLAlchemy

```
session = Session()

try:
    ...
    session.commit()
except:
    session.rollback()
finally:
    session.close()
```

14. Оптимизация запросов и производительности

Основные методы

- использовать `joinedload()` и `selectinload()`;
- выбирать только нужные столбцы (`load_only()`);
- использовать индексы;
- избегать N+1 запросов;
- применять пагинацию (`limit, offset`);
- выполнять массовые операции (`bulk_insert_mappings`);
- использовать пул соединений.

Профилирование

- включить логирование SQL:

```
engine = create_engine(url, echo=True)
```

- анализировать время выполнения запросов.

Кэширование

- кэшировать часто используемые данные;
- использовать Redis, Memcached.

Схема БД

- правильно выбирать типы данных;
- создавать индексы;
- использовать внешние ключи;
- нормализовать структуру таблиц.

15. Что такое Flask

Определение

Flask — лёгкий веб-фреймворк для Python.

Почему микрофреймворк

Не содержит встроенных:

- ORM;
- системы авторизации;
- форм;
- панели администратора.

Все дополнительные возможности подключаются расширениями.

Особенности

- минимализм;
- простая маршрутизация;
- встроенный сервер разработки;
- шаблонизатор Jinja2;
- поддержка расширений.

Области применения

- REST API;
- веб-приложения;
- микросервисы;
- учебные проекты.

16. Создание Flask-приложения и маршрутизация

Минимальное приложение:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "Hello"

@app.route("/about")
def about():
    return "About"

if __name__ == "__main__":
    app.run()
```

Маршрутизация

Декоратор `@app.route()` связывает URL с функцией.

Передача параметров:

```
@app.route("/user/<name>")
def user(name):
    return name
```

17. Обработка GET и POST

GET

Получение данных.

```
@app.route("/")
def index():
    return "GET"
```

POST

Отправка данных.

```
from flask import request

@app.route("/form", methods=["GET", "POST"])
def form():
    if request.method == "POST":
        name = request.form["name"]
        return name
    return "<form method='post'><input name='name'></form>"
```

Получение данных

- `request.args` — GET-параметры;
- `request.form` — данные формы;
- `request.json` — JSON.

18. Jinja2

Определение

Jinja2 — шаблонизатор, используемый Flask для генерации HTML.

Передача данных:

```
return render_template("index.html", name="Ivan")
```

Шаблон:

```
<h1>{{ name }}</h1>
```

Цикл

```
{% for user in users %}
    <p>{{ user.name }}</p>
{% endfor %}
```

Условие

```
{% if admin %}
    Admin
{% endif %}
```

19. Обработка форм и валидация

Flask-WTF

```
from flask_wtf import FlaskForm
from wtforms import StringField
from wtforms.validators import DataRequired

class UserForm(FlaskForm):
    name = StringField(validators=[DataRequired()])
```

Проверка:

```
form = UserForm()

if form.validate_on_submit():
    ...
```

Самостоятельная валидация

```
name = request.form.get("name")

if not name:
    return "Ошибка"
```

Ошибки выводятся через `form.errors` или вручную.

20. Обработка ошибок

Создание обработчика:

```
@app.errorhandler(404)
def not_found(error):
    return "Страница не найдена", 404
```

Ошибка сервера:

```
@app.errorhandler(500)
def server_error(error):
    return "Ошибка сервера", 500
```

Можно возвращать HTML-шаблон:

```
return render_template("404.html"), 404
```

21. Flask-расширения

Назначение

Расширения добавляют функциональность без изменения ядра Flask.

Примеры:

- **Flask-SQLAlchemy** — работа с БД;
- **Flask-Login** — аутентификация;
- **Flask-WTF** — формы;
- **Flask-Migrate** — миграции.

Подключение

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy(app)
```

или

```
db = SQLAlchemy()
db.init_app(app)
```

Преимущества

- сокращают объём кода;
- стандартизируют разработку;
- упрощают интеграцию компонентов.

22. Сессии во Flask

Определение

Session — механизм хранения данных пользователя между запросами.

Использование

```
from flask import session

session["user"] = "Ivan"
name = session.get("user")
session.pop("user")
```

Хранение

По умолчанию:

- данные находятся в **подписанных cookie**;
- защита осуществляется с помощью `SECRET_KEY`.

```
app.config["SECRET_KEY"] = "secret"
```

Защита

- использовать сложный `SECRET_KEY`;
- передавать cookie только по HTTPS;
- устанавливать `HttpOnly` и `Secure`;
- не хранить в сессии конфиденциальные данные.

23. Статические файлы во Flask

Структура

Статические файлы хранятся в папке:

```
/static
  /css
  /js
  /images
```

Подключение в шаблонах

Используется `url_for('static', filename=...)`:

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
<script src="{{ url_for('static', filename='js/app.js') }}"></script>

```

Особенности

- Flask автоматически обслуживает `/static`;
- не требует отдельной настройки в разработке;
- в продакшене обычно выносятся в Nginx/Apache.

24. RESTful API во Flask

Принципы REST

- ресурсы представлены URL;
- использование HTTP методов:
 - GET — получить
 - POST — создать
 - PUT/PATCH — обновить
 - DELETE — удалить
- обмен данными в JSON.

Пример API endpoint

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/api/user", methods=["POST"])
def create_user():
    data = request.get_json()
    return jsonify({
        "name": data["name"],
        "status": "created"
    })
```

Получение JSON

```
data = request.get_json()
```

Ответ JSON

```
return jsonify({"key": "value"})
```

Особенности

- обязательное использование JSON;
- отсутствие состояния (stateless);
- корректные HTTP-коды (200, 201, 400, 404);
- валидация входных данных обязательна.

25. Аутентификация и авторизация во Flask

Flask-Login (сессии)

Используется для веб-приложений:

- хранит пользователя в session;
- работает через cookie.

```
from flask_login import LoginManager, login_user, login_required

login_user(user)
```

Проверка доступа:

```
@login_required
def profile():
    return "Profile"
```

Flask-JWT (API)

Используется для REST API:

- токены JWT;
- stateless аутентификация.

```
from flask_jwt_extended import create_access_token

token = create_access_token(identity=user.id)
```

Сравнение

Flask-Login	Flask-JWT
Session-based	Token-based
Web приложения API	
Cookie	HTTP Header

Внешние сервисы

- OAuth2 (Google, GitHub);
- OpenID Connect;
- SSO системы.

26. Тестирование Flask-приложений

Юнит-тесты

Тестируют отдельные функции:

```
def test_add():  
    assert add(2, 3) == 5
```

Функциональные тесты

Тестируют маршруты:

```
def test_home(client):  
    response = client.get("/")  
    assert response.status_code == 200
```

Flask test client

```
import pytest  
from app import app  
  
@pytest.fixture  
def client():  
    return app.test_client()
```

Пример запроса

```
response = client.post("/api/user", json={"name": "Ivan"})
```

Особенности

- используется pytest;
- изоляция тестовой среды;
- тестовая БД;
- проверка HTTP-кодов и JSON.

27. Django

Определение

Django — высокоуровневый веб-фреймворк Python с полной комплектацией.

Отличия

- Flask — минимализм;
- Django — "batteries included" (всё встроено).

Преимущества

- встроенная ORM;
- админ-панель;
- система аутентификации;
- миграции;
- безопасность (CSRF, XSS защита);
- масштабируемость.

Философия

- быстрое создание приложений;
- DRY (Don't Repeat Yourself);
- строгая структура проекта.

28. Архитектура MVT в Django

Model

- работа с БД;
- описание таблиц;
- бизнес-данные.

```
class User(models.Model):  
    name = models.CharField(max_length=100)
```

View

- бизнес-логика;
- обработка запросов;
- возвращает response.

```
def index(request):  
    return HttpResponse("Hello")
```

Template

- HTML-шаблоны;
- отображение данных.

```
<h1>{{ name }}</h1>
```

Взаимодействие

Request → View → Model → View → Template → Response

29. Создание Django проекта и приложения

Создание проекта

```
django-admin startproject myproject
```

Запуск сервера

```
python manage.py runserver
```

Создание приложения

```
python manage.py startapp appname
```

Регистрация приложения

```
INSTALLED_APPS = [  
    "appname"  
]
```

30. Маршруты в Django

urls.py

Файл маршрутизации.

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path("", views.index),  
]
```

Подключение приложений

```
from django.urls import include  
  
urlpatterns = [  
    path("app/", include("app.urls"))  
]
```

Параметры URL

```
path("user/<int:id>/", views.user)
```

View

```
def user(request, id):  
    return HttpResponse(str(id))
```

31. Модели и миграции Django

Модель

```
class Product(models.Model):  
    name = models.CharField(max_length=100)  
    price = models.IntegerField()
```

Поля

- CharField — строки;
- IntegerField — числа;
- DateTimeField — дата/время;
- ForeignKey — связь.

Миграции

Создание:

```
python manage.py makemigrations
```

Применение:

```
python manage.py migrate
```

Суть миграций

- преобразование моделей в SQL;
- контроль версий структуры БД;
- автоматическое обновление схемы.

32. Django ORM

Возможности

- работа с БД через Python;
- автоматическая генерация SQL;
- поддержка связей;
- QuerySet API.

Примеры

```
User.objects.all()  
User.objects.filter(name="Ivan")
```

Преимущества

- защита от SQL-инъекций;
- читаемый код;
- кэширование QuerySet;
- ленивые запросы;
- кросс-БД совместимость.

Отличие от SQL

ORM	SQL
Python-объекты	строки SQL
безопаснее	гибче
проще	сложнее

33. Django Admin

Назначение

Админ-панель для управления данными.

Подключение модели

```
from django.contrib import admin
from .models import Product

admin.site.register(Product)
```

Возможности

- CRUD через интерфейс;
- фильтры;
- поиск;
- сортировка.

Кастомизация

```
class ProductAdmin(admin.ModelAdmin):
    list_display = ("name", "price")
    search_fields = ("name",)

admin.site.register(Product, ProductAdmin)
```

Особенности

- автоматически создаётся;
- требует только регистрации моделей;
- защищена системой аутентификации.

34. Form и ModelForm в Django

Form

Используется для ручной валидации данных.

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
```

ModelForm

Связан с моделью БД.

```
from django.forms import ModelForm
from .models import User

class UserForm(ModelForm):
    class Meta:
        model = User
        fields = ["name", "email"]
```

View (GET/POST)

```
from django.shortcuts import render

def contact(request):
    if request.method == "POST":
        form = ContactForm(request.POST)
        if form.is_valid():
            name = form.cleaned_data["name"]
            return render(request, "success.html")
    else:
        form = ContactForm()

    return render(request, "form.html", {"form": form})
```

Валидация

- `is_valid()` — проверка;
- `cleaned_data` — очищенные данные;
- `errors` — ошибки формы.

35. Аутентификация и авторизация в Django

User модель

- встроенная `django.contrib.auth.models.User`;
- содержит `username`, `password`, `email`, `groups`, `permissions`.

Login

```
from django.contrib.auth import login

login(request, user)
```

Logout

```
from django.contrib.auth import logout

logout(request)
```

Проверка авторизации

```
request.user.is_authenticated
```

Декораторы доступа

```
from django.contrib.auth.decorators import login_required

@login_required
def profile(request):
    ...
```

Права доступа

- Permissions (add, change, delete);
- Groups;
- Custom permissions.

```
user.has_perm("app.change_model")
```

Сессии

- хранят `user_id`;
- связаны с `cookie`;
- управляются `middleware`.

36. Django Middleware

Определение

Middleware — слой между request и response.

Функции

- обработка запросов;
- изменение response;
- логирование;
- безопасность;
- кэширование;
- проверка прав.

Пример

```
class SimpleMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Request")
        response = self.get_response(request)
        print("Response")
        return response
```

Поток

Request → Middleware → View → Middleware → Response

37. Django Templates

Синтаксис

```
<h1>{{ title }}</h1>
```

Условия

```
{% if user.is_authenticated %}  
    Hello  
{% endif %}
```

Циклы

```
{% for item in list %}  
    {{ item }}  
{% endfor %}
```

Фильтры

```
{{ name|upper }}  
{{ date|date:"Y-m-d" }}
```

Наследование

Базовый шаблон:

```
{% block content %}{% endblock %}
```

Дочерний:

```
{% extends "base.html" %}  
{% block content %}  
    Page content  
{% endblock %}
```

Теги

- {% if %}
- {% for %}
- {% url %}
- {% include %}

38. FBV vs CBV

Function-Based Views (FBV)

```
def index(request):  
    return HttpResponse("Hello")
```

Плюсы:

- простота;
- контроль логики;
- легко читать.

Минусы:

- мало переиспользования;
- дублирование кода.

Class-Based Views (CBV)

```
from django.views import View  
  
class IndexView(View):  
    def get(self, request):  
        return HttpResponse("Hello")
```

Плюсы:

- наследование;
- переиспользование;
- встроенные generic views.

Минусы:

- сложнее;
- "магия" Django.

Когда использовать

- FBV → простые задачи;
- CBV → CRUD, сложная логика.

39. Static и Media файлы в Django

Static

```
STATIC_URL = "/static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

Media

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Раздача в dev

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Отличия

Static	Media
код сайта	пользовательские файлы
CSS/JS	загрузки пользователей

Production

- Nginx/Apache;
- CDN для static/media.

40. Конфигурация Django (окружения)

Подход

Разделение настроек:

- dev;
- test;
- prod.

Переменные окружения

```
import os

SECRET_KEY = os.environ.get("SECRET_KEY")
DEBUG = os.environ.get("DEBUG") == "True"
```

Best practices

- не хранить секреты в коде;
- использовать `.env`;
- разделять `settings.py`;
- использовать `django-environ`.

Структура

```
settings/  
    base.py  
    dev.py  
    prod.py
```

41. Безопасность Django

CSRF

- защита POST-запросов;
- токен в форме.

```
{% csrf_token %}
```

XSS

- авто-экранирование шаблонов;
- фильтр escape.

SQL Injection

- ORM защищает автоматически;
- нет прямых SQL строк.

Clickjacking

```
X_FRAME_OPTIONS = "DENY"
```

Дополнительно

- password hashing;
- secure cookies;
- middleware безопасности.

42. Django REST Framework (DRF)

Определение

DRF — библиотека для создания REST API на Django.

Сериализаторы

```
from rest_framework import serializers

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = "__all__"
```

ViewSet

```
from rest_framework.viewsets import ModelViewSet

class UserViewSet(ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
```

Router

```
from rest_framework.routers import DefaultRouter

router.register("users", UserViewSet)
```

Преимущества

- быстрый API;
- встроенная валидация;
- авторизация;
- pagination;
- удобные ViewSet.

43. Пагинация, фильтрация, сортировка

Пагинация

```
from django.core.paginator import Paginator

p = Paginator(User.objects.all(), 10)
page = p.get_page(1)
```

Фильтрация

```
User.objects.filter(name="Ivan")
```

Сортировка

```
User.objects.order_by("name")
```

Комбинация

```
User.objects.filter(active=True).order_by("-date")
```

DRF

- `filters.OrderingFilter;`
- `DjangoFilterBackend.`

44. Оптимизация Django

Кэширование

- per-view;
- template fragment;
- Redis/Memcached.

```
from django.views.decorators.cache import cache_page
```

Оптимизация ORM

- `select_related()` — JOIN;
- `prefetch_related()` — связанные данные;
- индексы в БД;
- избегать N+1.

CDN

- статические файлы;
- изображения;
- ускорение доставки.

Сервер

- Gunicorn / uWSGI;
- Nginx;
- балансировка нагрузки.

Дополнительно

- уменьшение запросов;
- lazy loading данных;
- профилирование SQL;
- оптимизация миграций.